



دانشگاه صنعتی امیر کبیر
(پلی تکنیک تهران)

Computer Architecture

Arithmetic for Computers

Afarghadan@aut.ac.ir

عظیم فرقدان



- 1 Introduction**
- 2 Addition and Subtraction**
- 3 Multiplication**
- 4 Division**

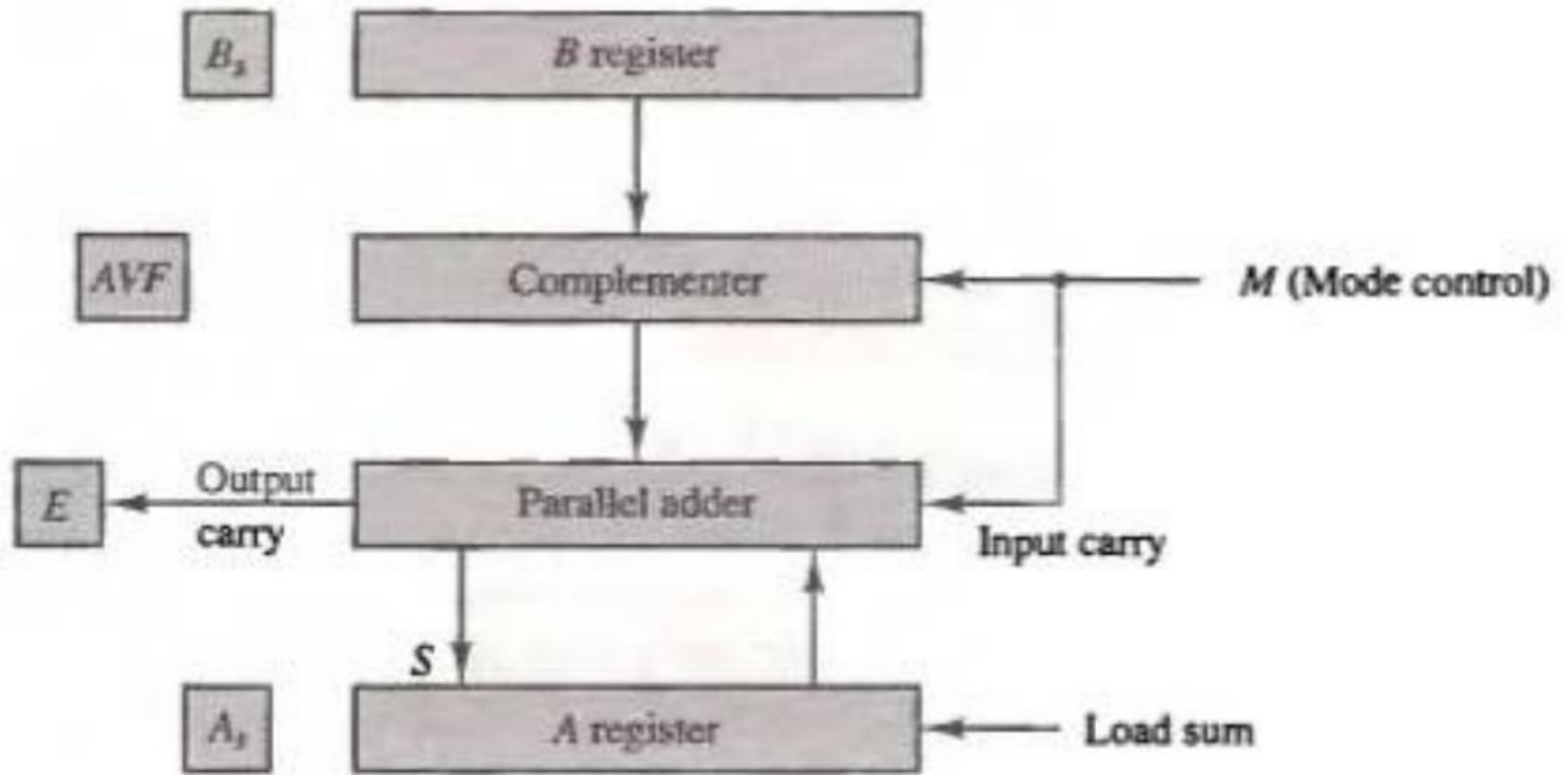
- Arithmetic instructions are **core** to computer processing, enabling operations like **addition, subtraction, multiplication, and division**.
- These operations use either **fixed**-point or **floating**-point number representations.
- Fixed-point numbers can represent integers or fractions; negative values are typically in **signed-complement** form.
- Arithmetic processor executes these instructions, often using data in processor **registers**.
- At a basic level, operations are performed in **signed-magnitude** representation, though **signed-2's complement** is more common in hardware.
- Algorithms and flowcharts guide the step-by-step execution of operations like binary addition and subtraction.

- Negative fixed-point binary numbers can be represented in
 - Signed-magnitude
 - Signed-1's complement
 - Signed-2's complement (**most commonly used in computers**)
- Floating-point operations typically use signed-magnitude

- There are **8** possible cases when adding/subtracting two signed binary numbers, based on the signs of the numbers and the operation
- If signs are **the same**
 - Add magnitudes, use the common sign
- If signs **differ**
 - Subtract smaller magnitude from larger
 - Use the sign of the larger magnitude
- **The logic is reversed for subtraction—use addition rules when signs differ**

Operation	Add Magnitudes	Subtract Magnitudes		
		When $A > B$	When $A < B$	When $A = B$
$(+A) + (+B)$	$+(A + B)$			
$(+A) + (-B)$		$+(A - B)$	$-(B - A)$	$+(A - B)$
$(-A) + (+B)$		$-(A - B)$	$+(B - A)$	$+(A - B)$
$(-A) + (-B)$	$-(A + B)$			
$(+A) - (+B)$		$+(A - B)$	$-(B - A)$	$+(A - B)$
$(+A) - (-B)$	$+(A + B)$			
$(-A) - (+B)$	$-(A + B)$			
$(-A) - (-B)$		$-(A - B)$	$+(B - A)$	$+(A - B)$

- To implement the two-arithmetic operation with hardware, it is necessary that the two numbers be **stored in register**
- Let A and B be two register that holds the magnitude of the numbers, and A1 and B1 be two flip-flop that holds the corresponding signs
- The result of the operation is transfer to **third** register
- In hardware implementation
 - First, a **parallel-adder** is needed to perform the microoperation **$A+B$**
 - Second, a **comparator circuit** is needed to establish if **$A > B$, $A = B$, or $A < B$**
 - Third, **two parallel-subtractor** circuits are needed to perform the micro-operations **$A - B$ and $B - A$**



- It consists of register A and B and sign flip-flop A1 and B1
- **Subtraction** is done by adding A and 2's complement of B
 - The output carry is transferred to flip-flops **E**, where it can be checked to determine to relative magnitude of the two numbers
 - The add overflow flip-flop AVF holds the overflow bit when A and B are added

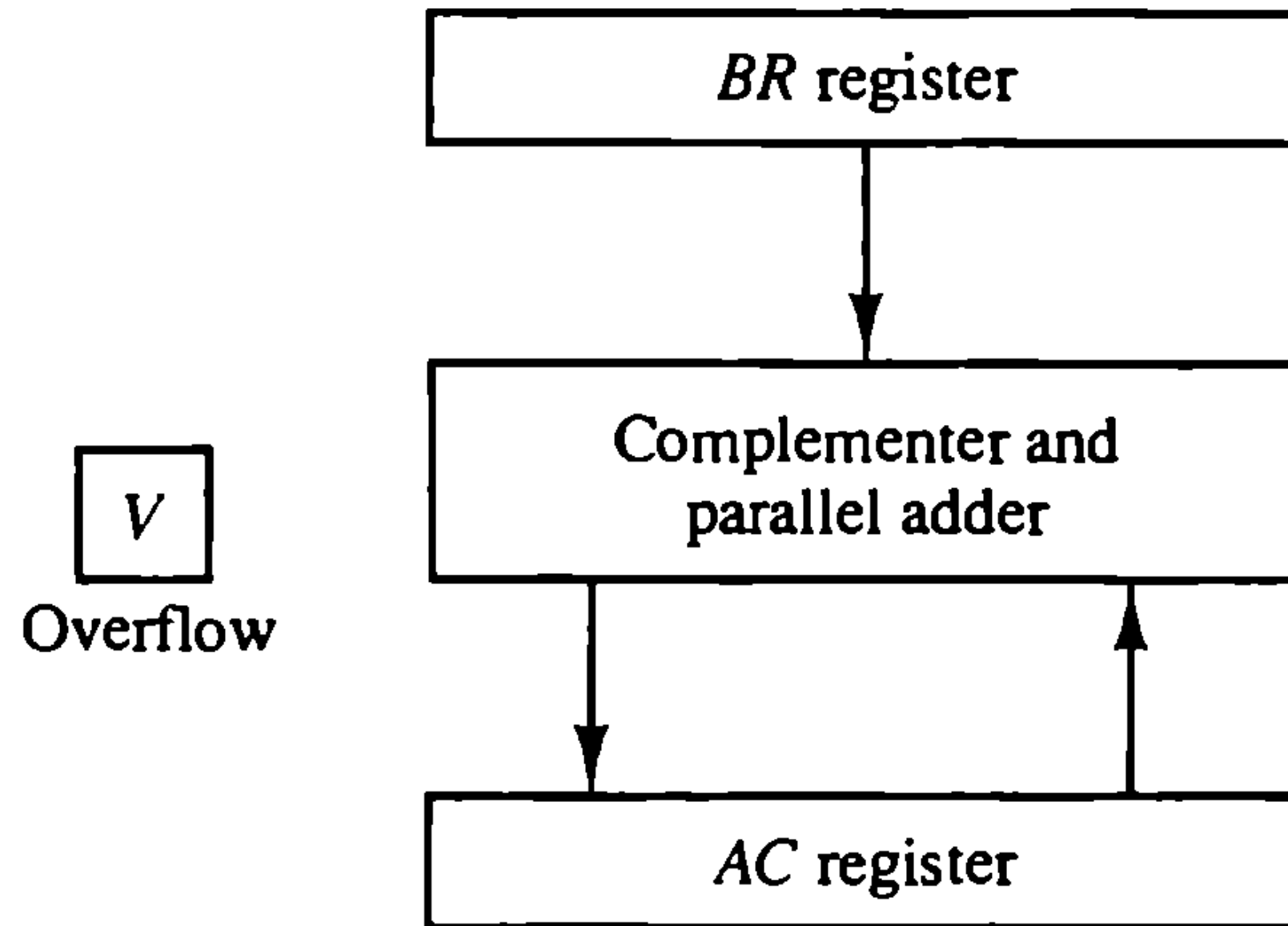
- **Addition** of A and B is done through the parallel adder (consists of full-adder)
 - The **S (Sum)** output of the adder is applied to the input of the **A** register
 - The **complementor** (consists of ex-OR gates) provides an output of B or the complement of B depending on the state of the mode control M
 - When **M=0**, the output of B is transferred to the adder, the input carry is 0, and the output of the adder is equal to the sum $A+B$
 - When **M=1**, the 1's complement of B is applied to the adder, the input carry is 1 and output $S=A+B+1$



۱۴۰۳ معماری کامپیوتر | محاسبات | عظیم فرقدان | بهمن

- The **leftmost** bit of a binary number represent the **sign** bit
 - **0** for positive
 - **1** for negative
- If the sign bit is 1, the entire number is represented in 2's complement form
- The addition of two number in signed complement form consists of adding the numbers with the sign bits treated the same as the order bits of the number
- A carry-out of the sign-bit position is **discarded**. The subtraction consists of first taking 2's complement of the subtrahend and then adding it to the minuend
- Overflow is detected when the output of the gate is equal to **1**

Figure 10-3 Hardware for signed-2's complement addition and subtraction.



2's complement (+/-) (Cont'd)

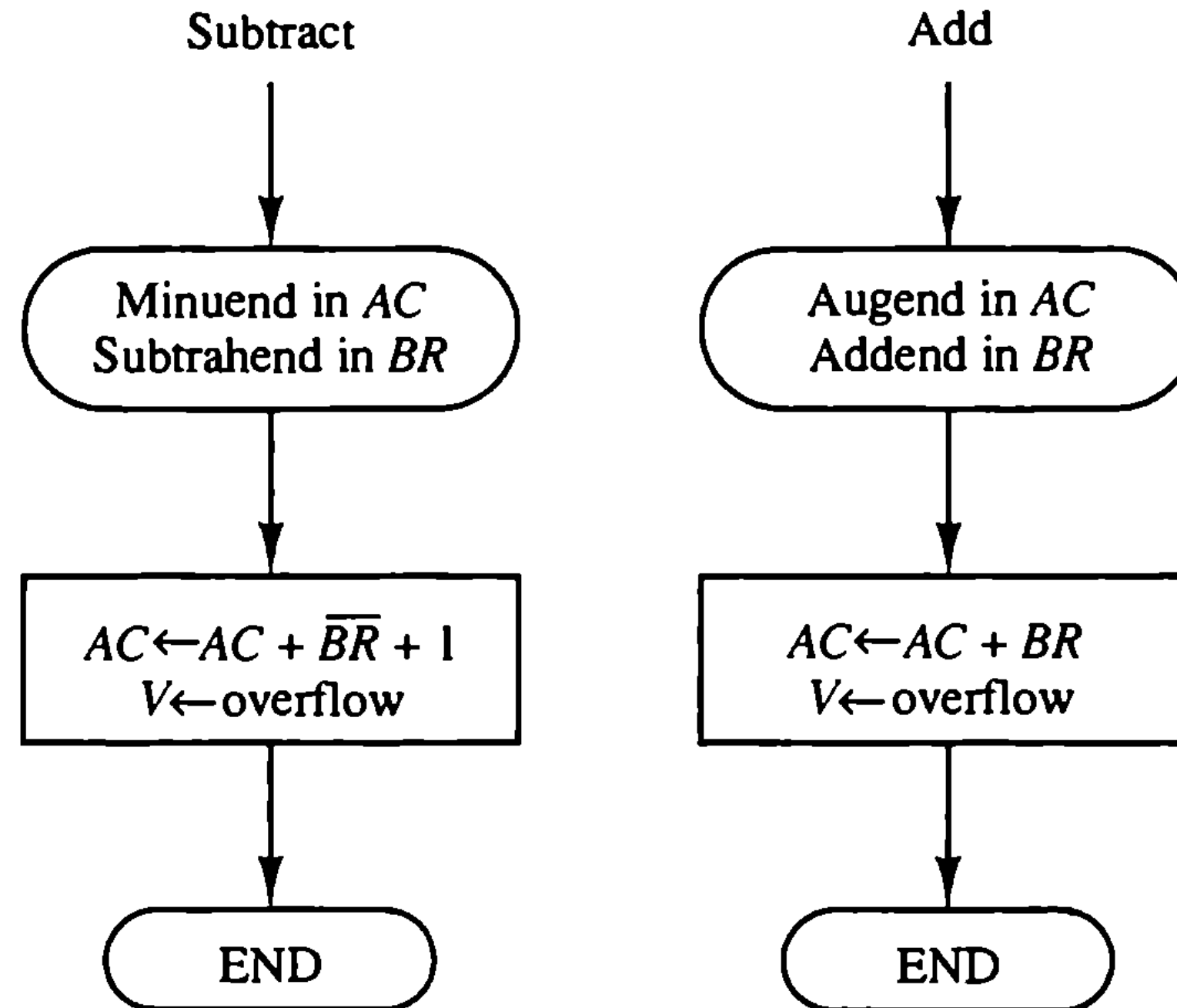


Figure 10-4 Algorithm for adding and subtracting numbers in signed-2's complement representation.

■ Example

23	10111	Multiplicand
<u>19</u>	<u>× 10011</u>	Multiplier
	10111	
	10111	
	00000	+
	00000	
	10111	
437	<u>110110101</u>	Product

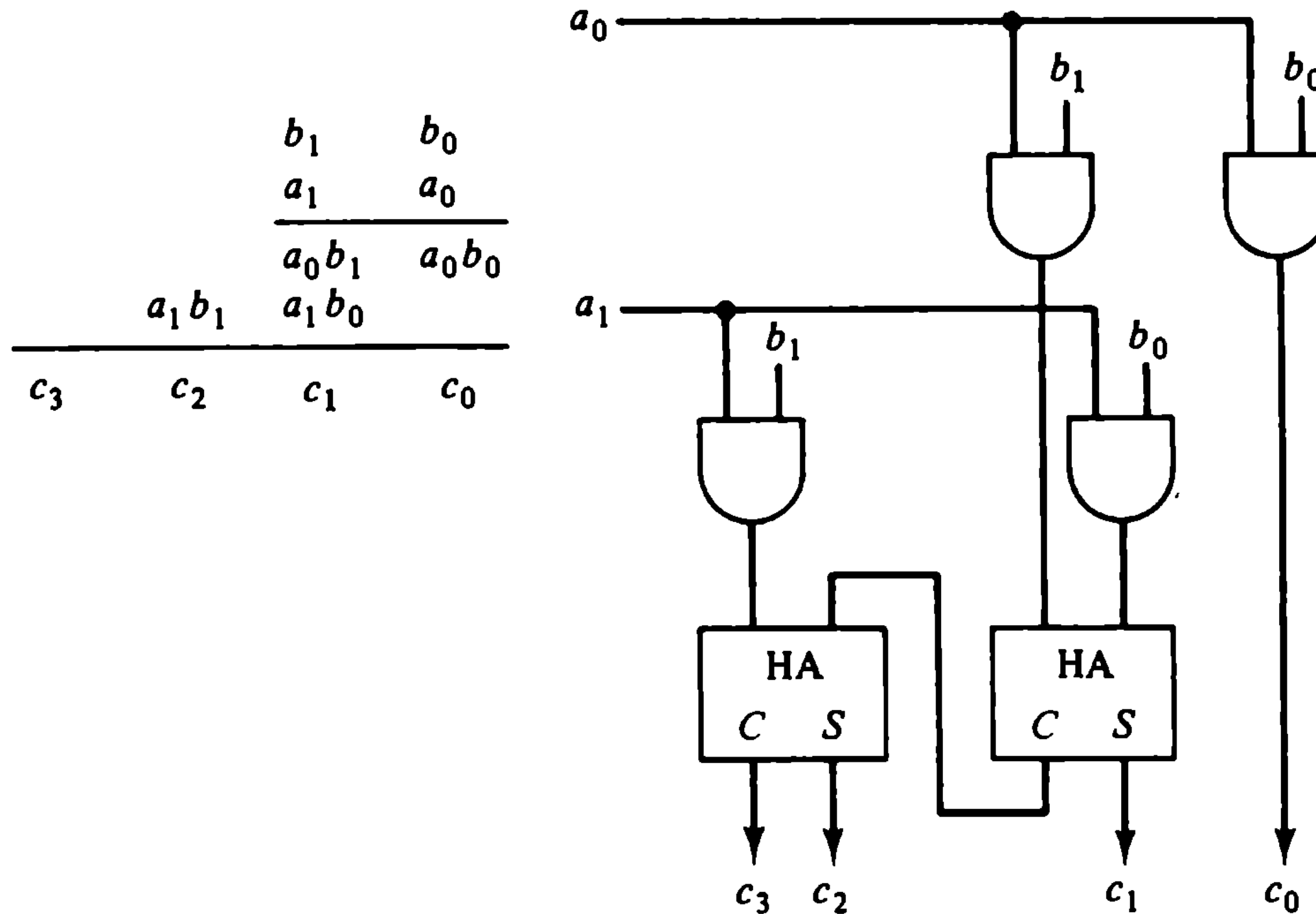
Multiplication Algorithm

- In multiplication algorithm, **successive** bits of the multiplier, least significant bit first
- If the **multiplier bit** is a 1, the multiplicand is **copied** down, otherwise zero are copied down
- The numbers copied down is successive lines are **shifted** one position to the left from the previous number. Finally, the numbers are **added**, and their sum forms the product
- The **sign** of the product is determined from the sign of the multiplicand and multiplier

- Checking the bits of the multiplier **one at a time** and forming partial products is a sequential operation that requires a sequence of add and shift micro operations
- The multiplication of two binary numbers can be done with one micro operation by means of a combinational circuit that forms the product bits all at once
- An array multiplier requires **a large number of gates, and integrated circuits**

Array Multiplier (Cont'd)

Figure 10-9 2-bit by 2-bit array multiplier.



Array Multiplier (Cont'd)

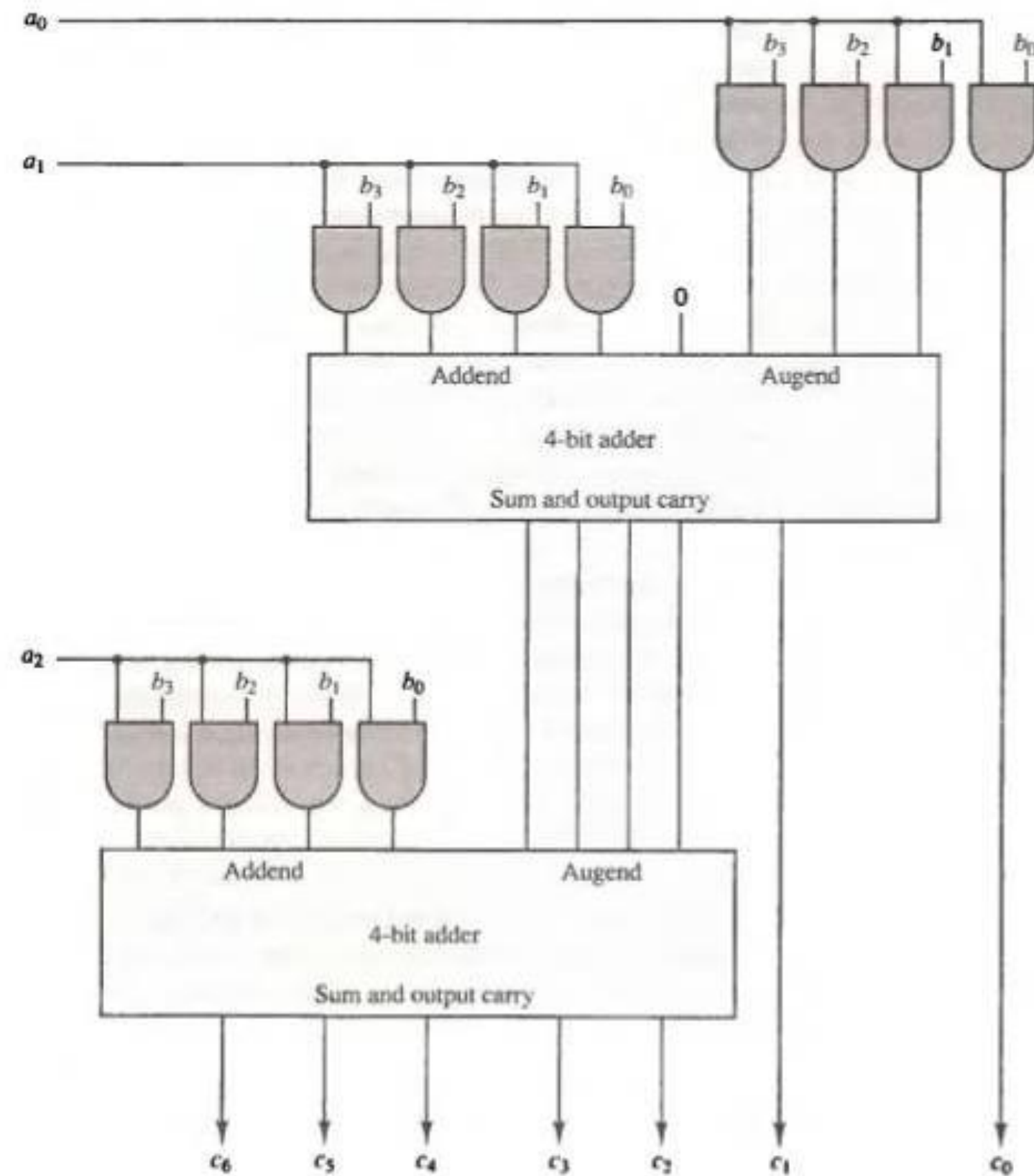
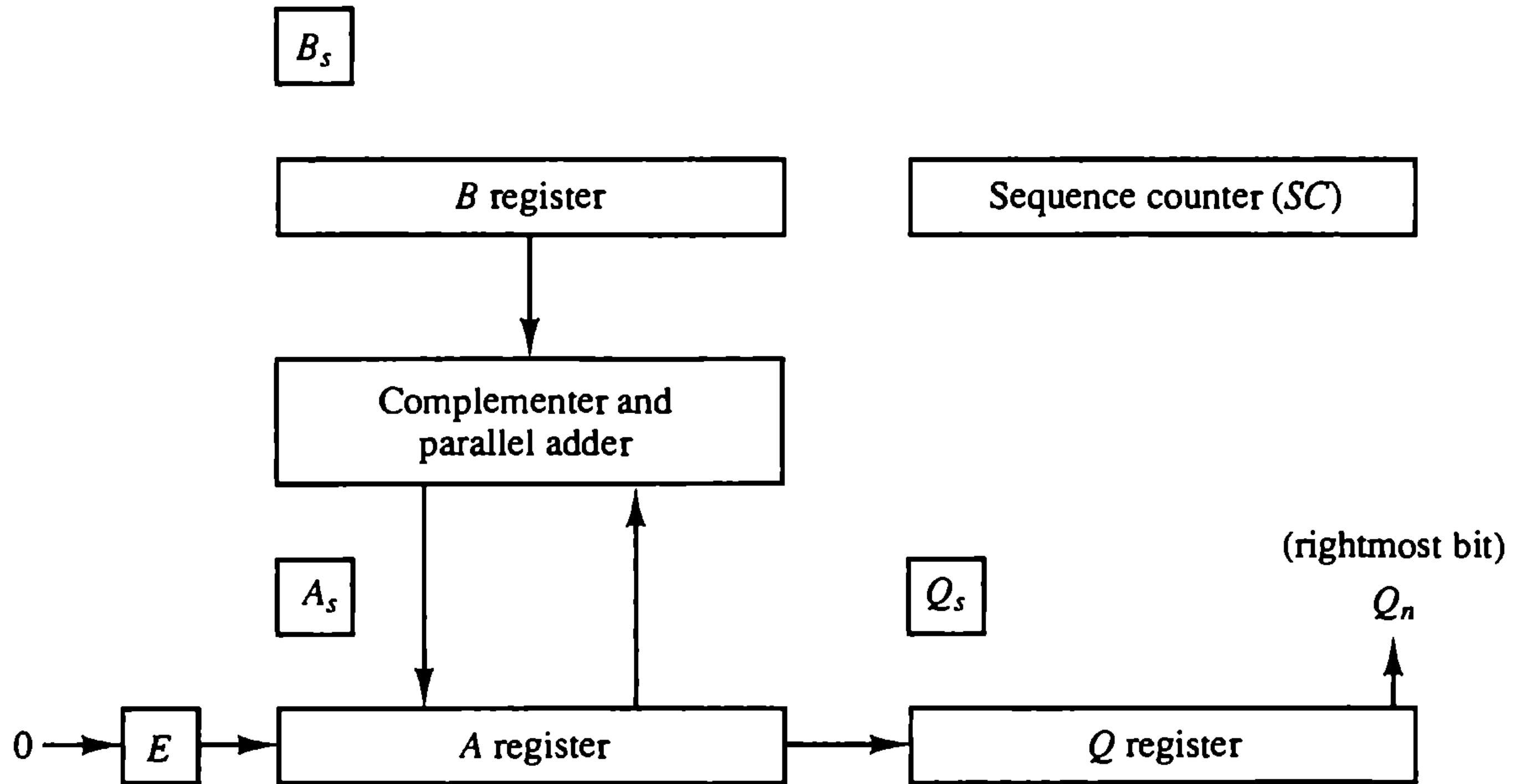


Figure 10-10 4-bit by 3-bit array multiplier.

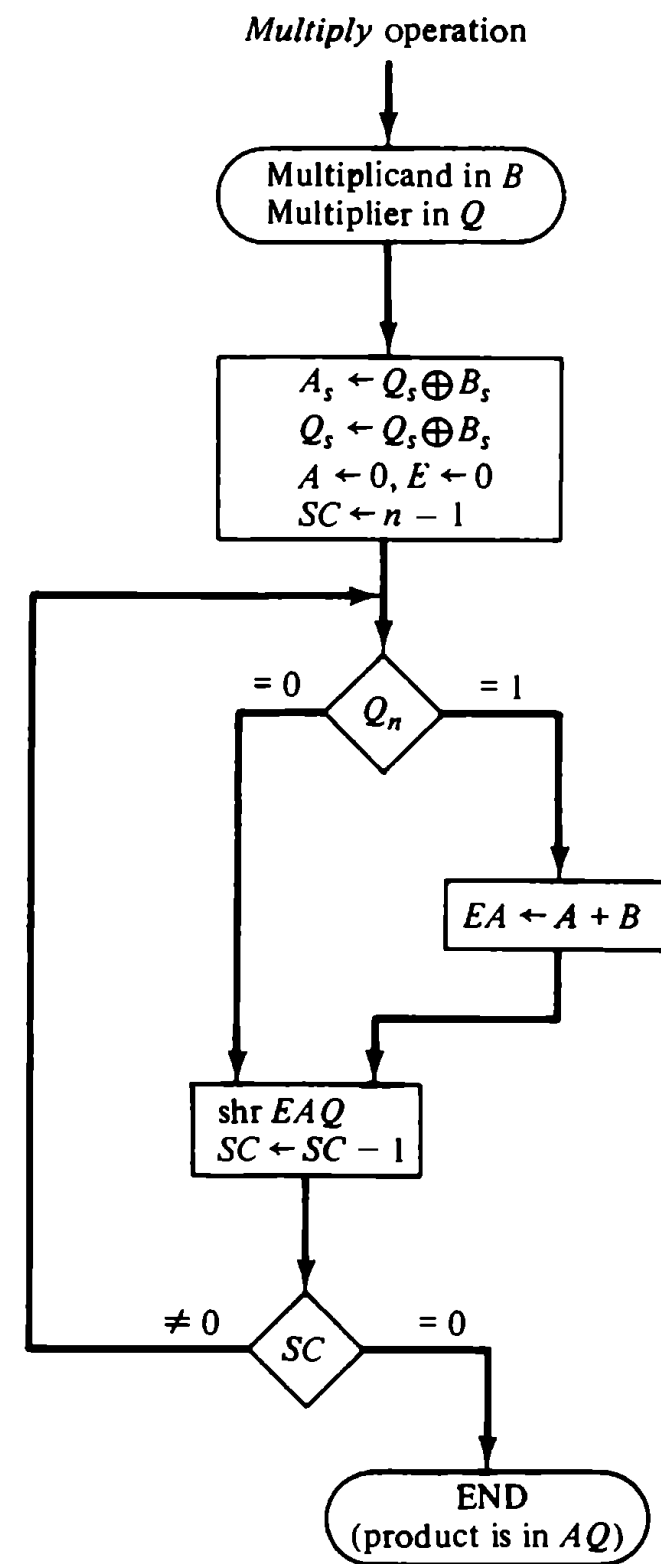
- First, instead of providing register to store and add simultaneously as many binary numbers as there are bits in the multiplier, it is convenient to provide an **adder** for the summation of only binary numbers and successively accumulated the partial products in registers
- Second, instead of shifting the multiplicand to the left, the partial product is shifted to the **right**, which results in leaving the partial product and the multiplicand in the required relative position
- Third when the corresponding bit of the multiplier is 0, there is no need to add all zeros to the partial product **since it will not alter its value**

Figure 10-5 Hardware for multiply operation.



- Initially, the multiplicand is in register B and the multiplier in Q
- The sum of A and B forms a partial product which is transferred to the **EA** register. Both partial product and multiplier are shifted to the right. This shift will be denoted by the statement shr EAQ to designate the right shift depicted
- **The least significant** bit of A is shifted into **the most significant** position of Q, the bit from E is shifted into the most significant position of A, and 0 is shifted into E
- After the shift, one bit of the partial product is shifted into Q, pushing the multiplier bits one position to the **right**
- Rightmost flip-flop in register Q, designated by Q_n , will hold the bit of the multiplier, which must be inspected next

Figure 10-6 Flowchart for multiply operation.



Booth Algorithm

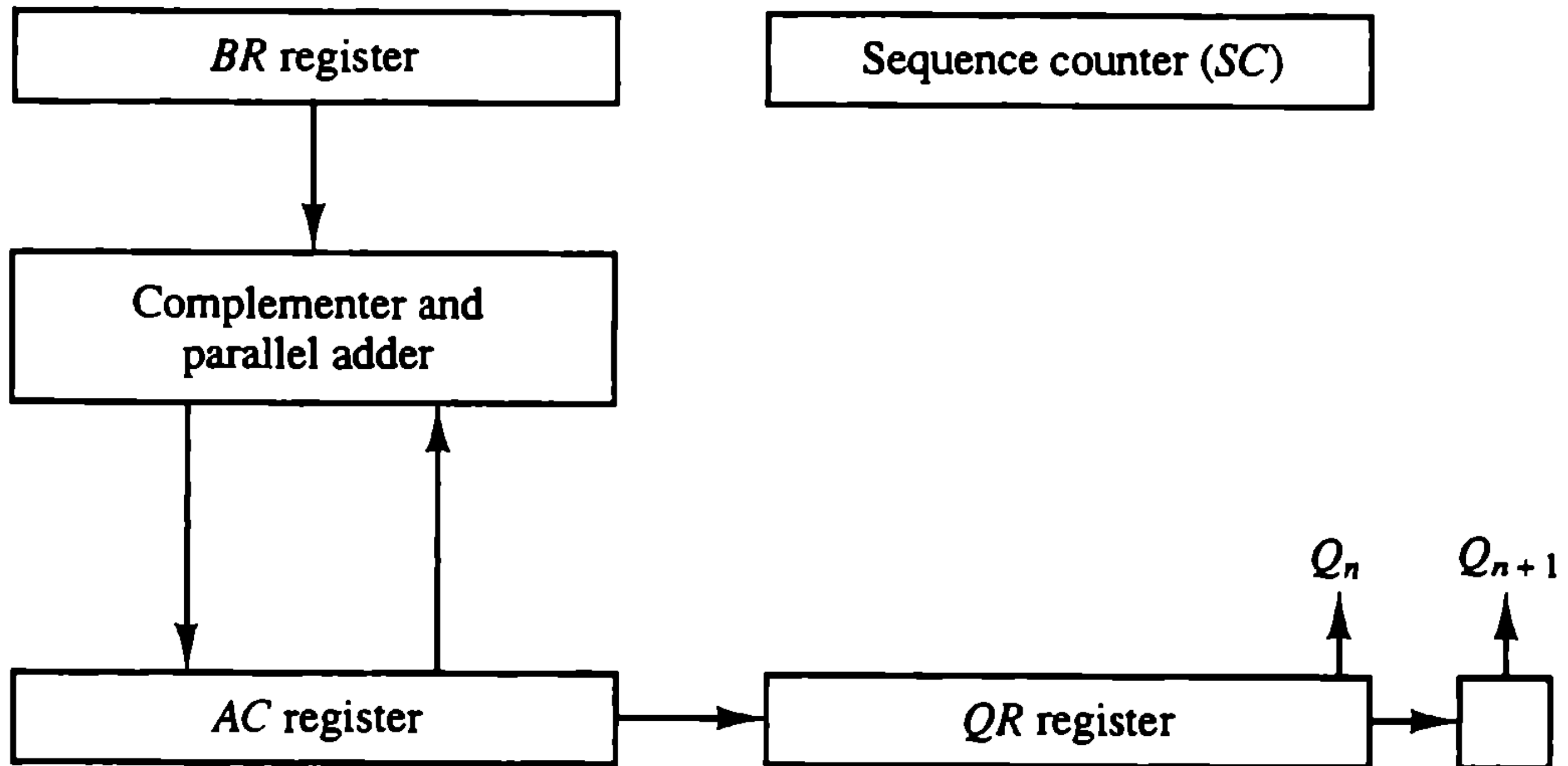
TABLE 10-2 Numerical Example for Binary Multiplier

Multiplicand $B = 10111$	E	A	Q	SC
Multiplier in Q	0	00000	10011	101
$Q_n = 1$; add B		<u>10111</u>		
First partial product	0	10111		
Shift right EAQ	0	01011	11001	100
$Q_n = 1$; add B		<u>10111</u>		
Second partial product	1	00010		
Shift right EAQ	0	10001	01100	011
$Q_n = 0$; shift right EAQ	0	01000	10110	010
$Q_n = 0$; shift right EAQ	0	00100	01011	001
$Q_n = 1$; add B		<u>10111</u>		
Fifth partial product	0	11011		
Shift right EAQ	0	01101	10101	000
Final product in $AQ = 0110110101$				

■ Rules

- The multiplicand is subtracted from partial product upon encountering first LSB 1 in a multiplier.
- 2. The multiplicand is added to the partial product upon encountering the first 0 (provided that there was a previous 1) in the multiplier.
- 3. The partial product does not change when the multiplier bit is identical to the previous multiplier bit.

Figure 10-7 Hardware for Booth algorithm.



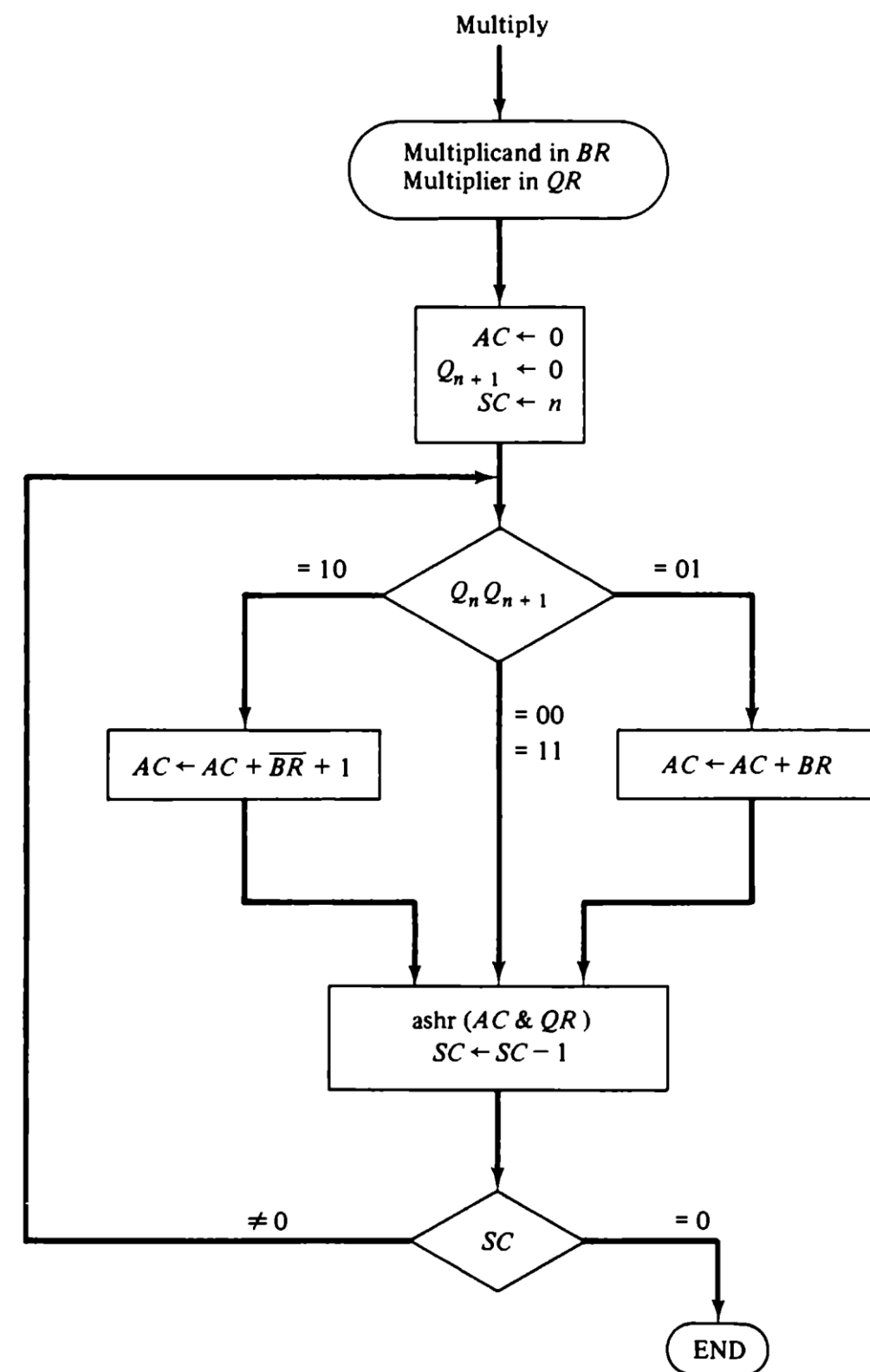


Figure 10-8 Booth algorithm for multiplication of signed-2's complement numbers.

Booth Multiplier (Cont'd)

TABLE 10-3 Example of Multiplication with Booth Algorithm

$Q_n Q_{n+1}$	$BR = 10111$ $\overline{BR} + 1 = 01001$	AC	QR	Q_{n+1}	SC
	Initial	00000	10011	0	101
1 0	Subtract BR	<u>01001</u> 01001			
	ashr	00100	11001	1	100
1 1	ashr	00010	01100	1	011
0 1	Add BR	<u>10111</u> 11001			
	ashr	11100	10110	0	010
0 0	ashr	11110	01011	0	001
1 0	Subtract BR	<u>01001</u> 00111			
	ashr	00011	10101	1	000

Division Algorithm

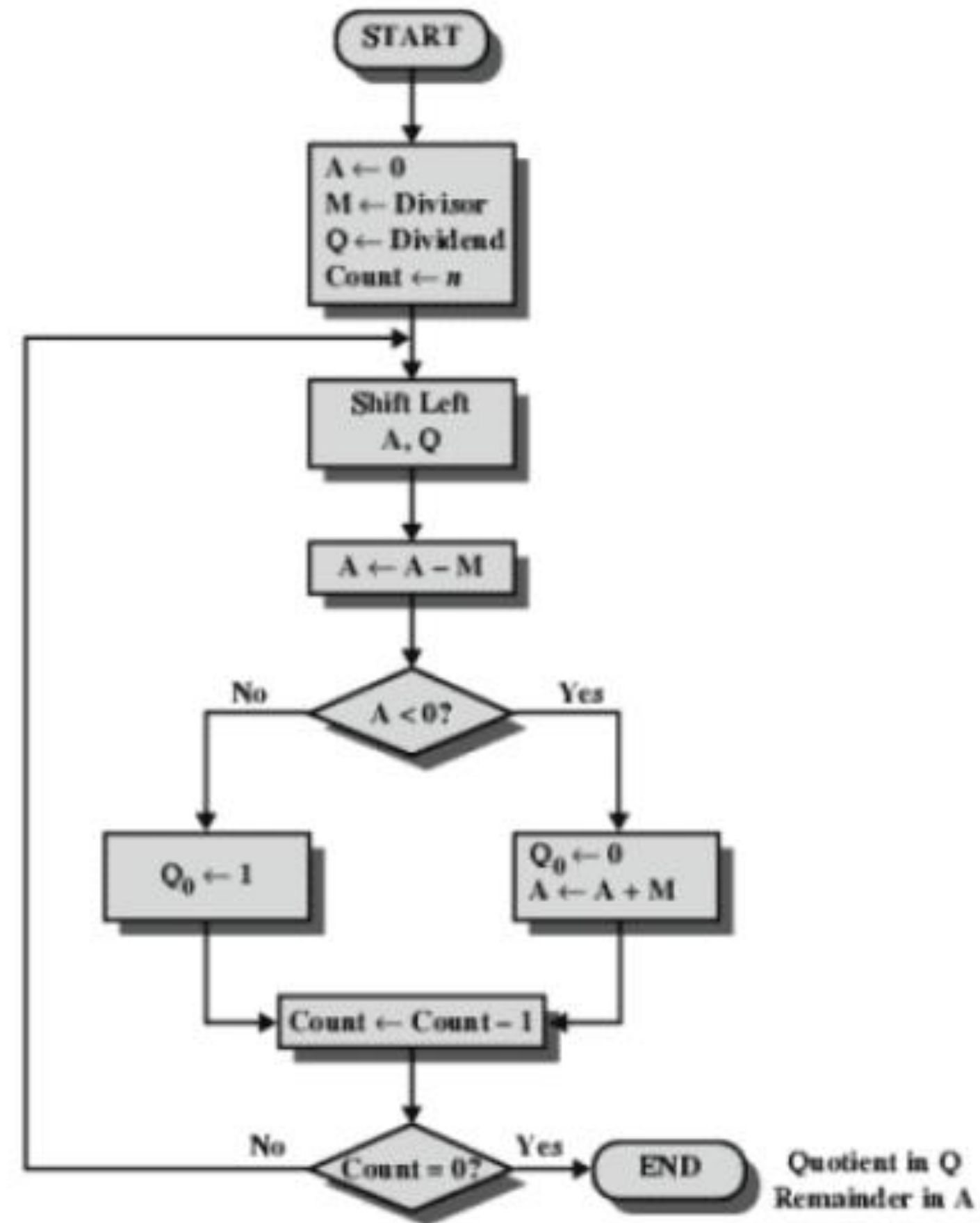
- Division of two fixed-point binary numbers in sign-magnitude representation is done with manually by processing of successive compare, shift and subtract operations

Figure 10-11 Example of binary division.

Divisor:	11010	Quotient = Q
$B = 10001$	$\overline{)0111000000}$	Dividend = A
	01110	5 bits of $A < B$, quotient has 5 bits
	011100	6 bits of $A \geq B$
	<u>-10001</u>	Shift right B and subtract; enter 1 in Q
	-010110	7 bits of remainder $\geq B$
	<u>--10001</u>	Shift right B and subtract; enter 1 in Q
	--001010	Remainder $< B$; enter 0 in Q ; shift right B
	---010100	Remainder $\geq B$
	<u>----10001</u>	Shift right B and subtract; enter 1 in Q
	----000110	Remainder $< B$; enter 0 in Q
	-----00110	Final remainder

- Step 1: Initialize A, Q and M registers to Zero, dividend and divisor respectively and counter to n where n is the number of bits in the dividend
- Step 2: Shift A, Q left on binary position
- Step 3: Subtract M from A placing answer back in A. If sign of A is 1, set Q0 to zero and add M back to A (restore A). If sign of A is 0, set Q0 to 1
- Step 4: Decrease counter; if counter > 0 , repeat process from step 2 else stop the process. The final remainder will be in A and Quotient will be in Q

Unsigned Binary Division Algorithm (Comparison)



Unsigned Binary Division Algorithm

Example : 15 / 4

Example: Divide 15 (1111) by 4 (0100)

A	Q	M	$\overline{M} + 1$	Count	Remarks
00000	1111	00100	11100	4	Initialization
00001 11101 00001	111□ 111□ 1110	- - -	- - -	- - 3	Shift Left A, Q Sub ($A \leftarrow A - M$) $Q_0 \leftarrow 0$, Add ($A \leftarrow A + M$)
00011 11111 00011	110□ 110□ 1100	- - -	- - -	- - 2	Shift Left A, Q Sub ($A \leftarrow A - M$) $Q_0 \leftarrow 0$, Add ($A \leftarrow A + M$)
00111 00011 00011	100□ 100□ 1001	- - -	- - -	- - 1	Shift Left A, Q Sub ($A \leftarrow A - M$) Set $Q_0 \leftarrow 1$
00111 00011 00011	001□ 001□ 0011	- - -	- - -	- - 0	Shift Left A, Q Sub ($A \leftarrow A - M$) Set $Q_0 \leftarrow 1$

Quotient in Q = 0011 = 3

Remainder in A = 00011 = 3

Division Algorithm (Restoring)

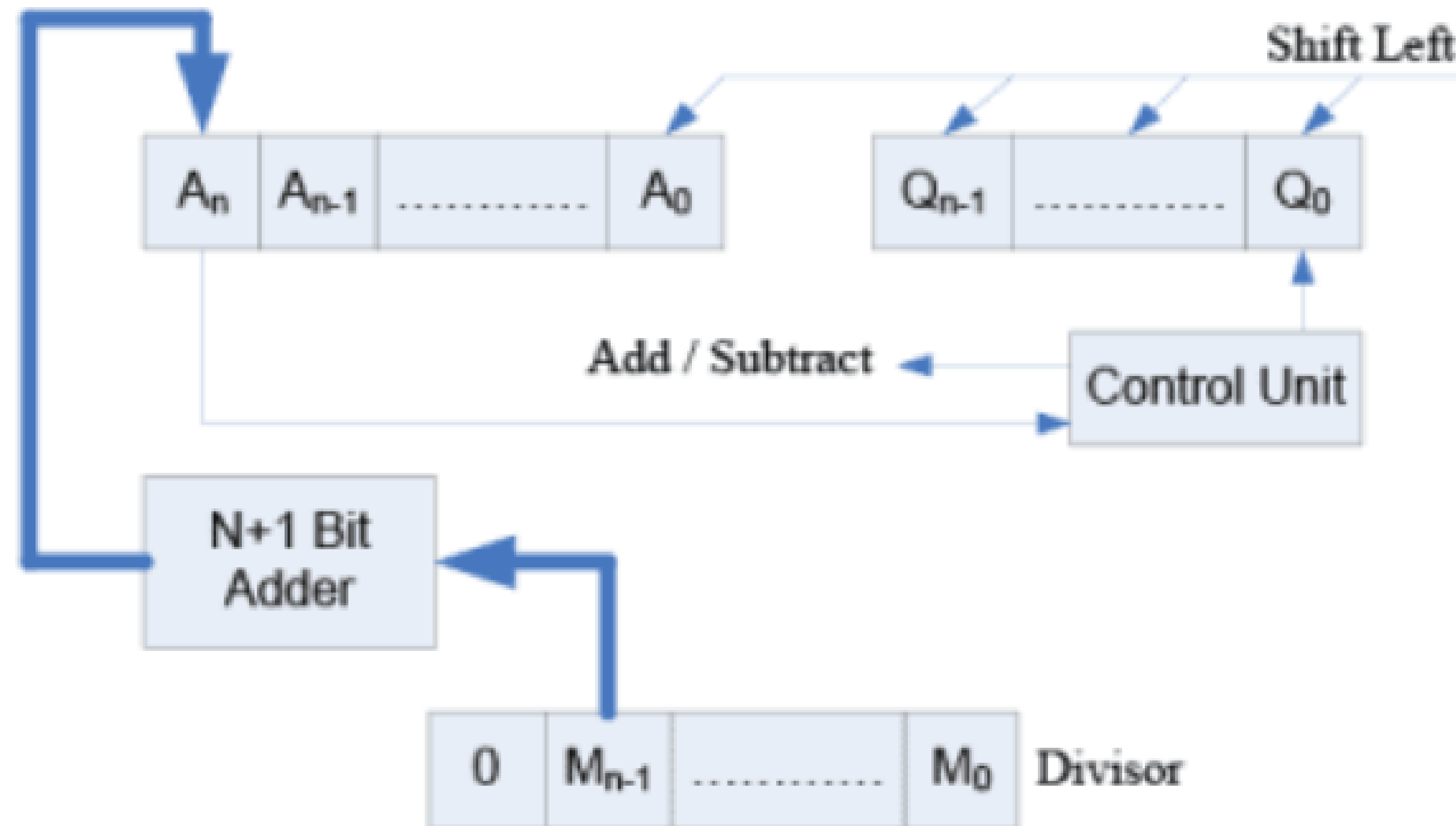


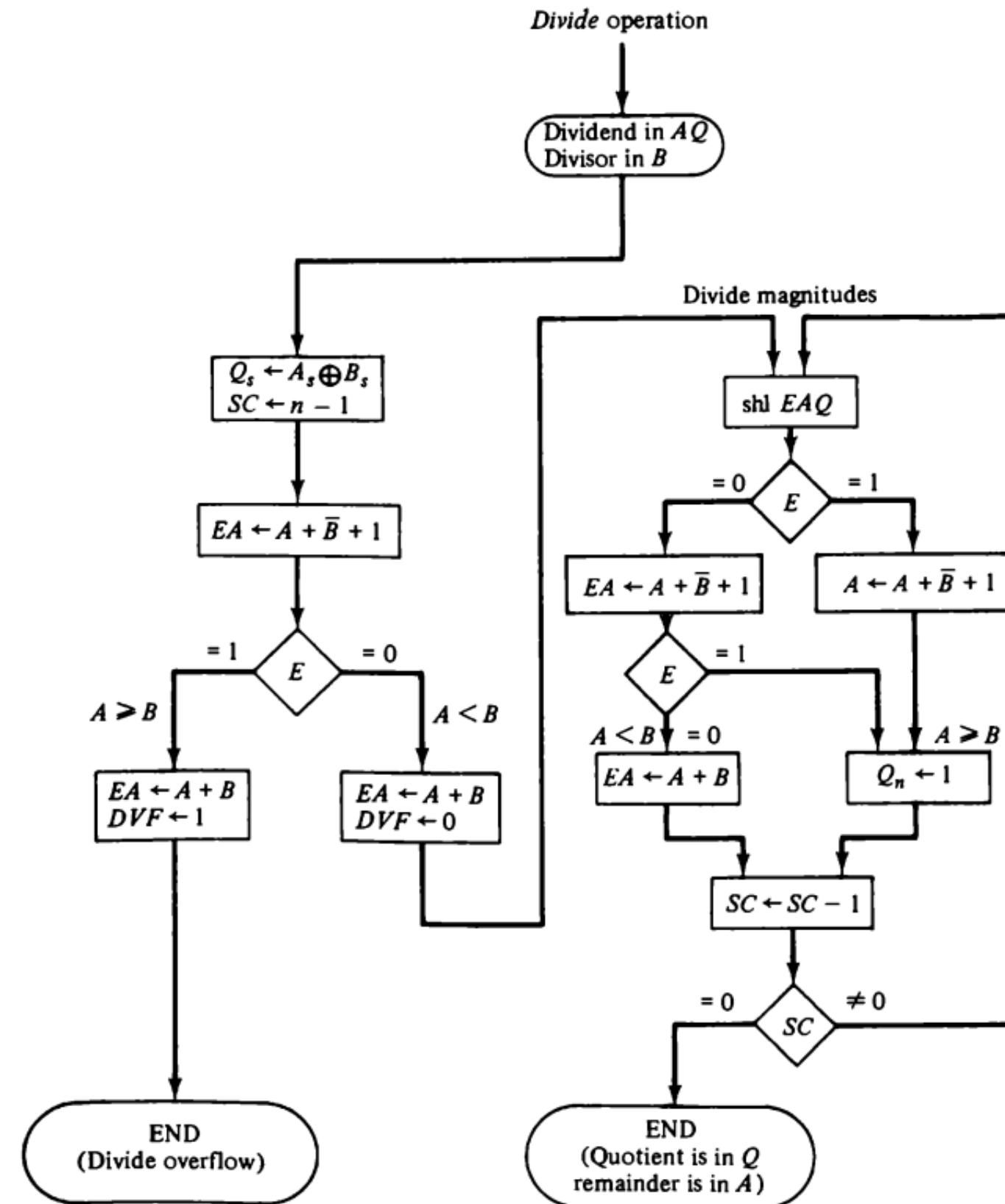
Fig.: Block Diagram of Division Operation

■ Rules

- First, the bits of the dividend are examined from left to right, until the set of bits examined represents a number greater than or equal to the divisor: this is referred to as the divisor being able to divide the number
- Until this event occurs, 0's are placed in the quotient from left to right
- When the event occurs, a 1 is placed in the quotient and the divisor is subtracted from the partial dividend. The result is referred to as a partial remainder
- The division follows a cyclic pattern. At each cycle, additional bits from the dividend are appended to the partial remainder until the result is greater than or equal to the divisor
- The divisor is subtracted from this number to produce a new partial remainder. The process continues until all the bits of the dividend are exhausted

Division Algorithm (Restoring)

Figure 10-13 Flowchart for divide operation.



- In a computer system, the division operation can lead to a quotient with an overflow because the registers cannot hold a number that exceeds the standard length. To understand this better, consider a system with a standard 5-bit register
- One register is used to hold the divisor and the other to hold the dividend. In case the quotient consists of 6 bits, 5 bits of the quotient will be stored in a 5-bit register. Therefore, the overflow bit needs a flip-flop to store the sixth bit
- The divide overflow condition occurs in case the high-order half bits of the dividend comprises a number that is greater than or equal to the divisor
- One other point that needs to be considered in the division is that it is advisable to avoid division by zero. The overflow condition is generally detected when a flip-flop is set. This flip-flop is known as DVF

Restoring Algorithm:

SHL EAQ
Sub

Restore:

Add
SHL EAQ
SUB

Non-Restoring Algorithm:

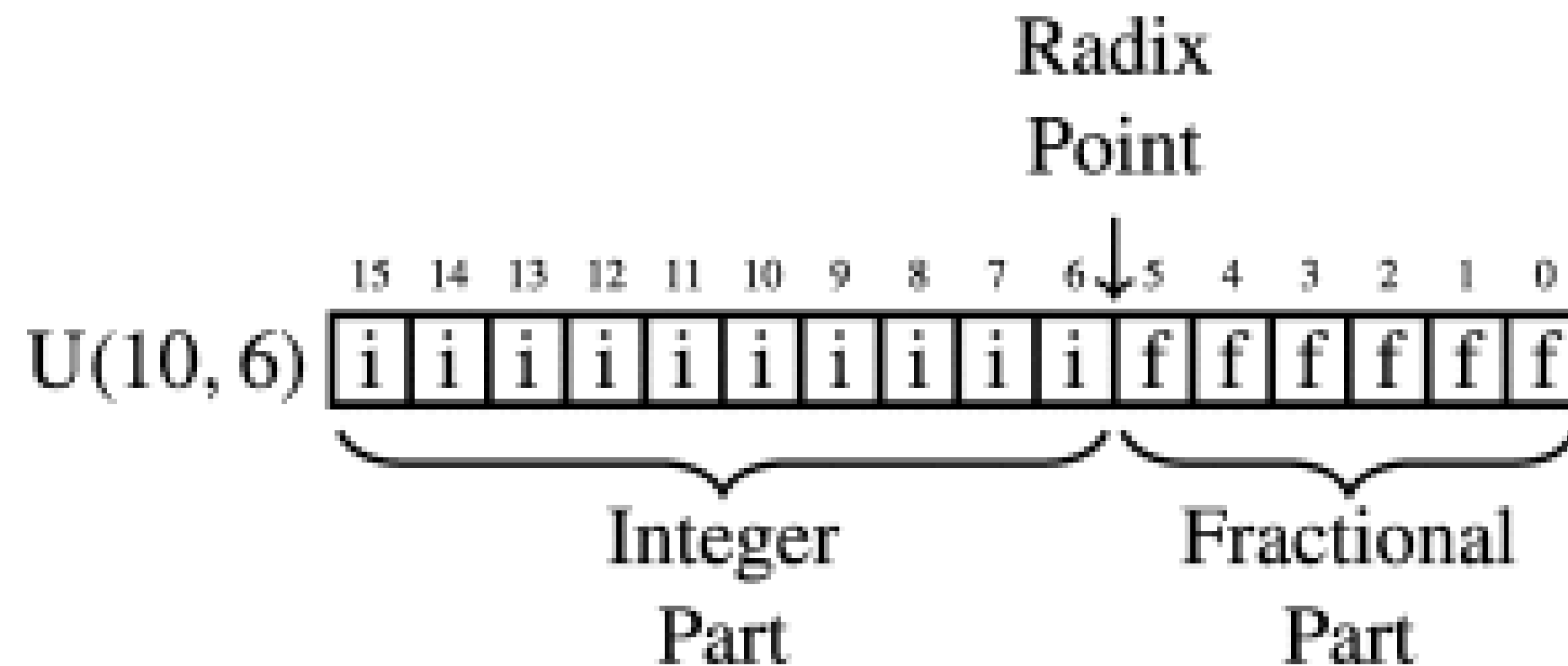
SHL EAQ
Sub

Restore:

SHL EAQ
Add

Fixed-Point Representation

- **A fixed number of fraction digits**
 - Fixed separator location
- Imagine an n-bit register
 - Usually the separator is located in the center
 - $n/2$ fraction bits
 - $n/2$ decimal bits
 - Is “*center*” the best location for the separator?



■ Practice

- For a fixed point register with n bits:
 - How many numbers can be represented?
 - Find the largest representable number
 - Find the smallest representable positive number (So called **Epsilon**)
 - Find the smallest distance between two consecutive representable numbers (So called **Resolution**)

- Representation for **non-integral numbers**
 - Including very small and very large numbers
- Like **scientific notation**
 - $-2.34 \times 10^{+56}$ (normalized)
 - $+0.002 \times 10^{-4}$ (not normalized)
 - $+987.02 \times 10^{+9}$ (not normalized)
- In binary
 - $\pm 1.xxxxxxx_2 \times 2^{yyyy}$
- Types **float** and **double** in C

Signe	Exponent	Mantissa or Significand
-------	----------	-------------------------

$$(-1)^S * 0.M * B^E$$

- The mantissa is usually stored normally, meaning that the most significant bit of the mantissa is opposite to zero.
- The number zero cannot be normalized because it has no non-zero digits.
- The normal mantissa value is at least 0.5 and at most less than one.

- The exponent itself is signed and can be stored using the 2's complement, 1's complement, and sign magnitudes.
- Exponents are usually stored in a fourth way called **Bias**.
- The **Bias is a positive number** that is added to the exponents and then stored.
- The bias system has the advantage that the **numbers are ordered** and the **smallest number has the smallest representation** and the **largest number has the largest representation**.

- The exponent itself is signed and can be stored using the 2's complement, 1's complement, and sign magnitudes.
- Exponents are usually stored in a fourth way called **Bias**.
- The **Bias is a positive number** that is added to the exponents and then stored.
- The bias system has the advantage that the **numbers are ordered** and the **smallest number has the smallest representation** and the **largest number has the largest representation**.
- If the exponent is e -bit, the bias value is 2^{e-1} .

- Assuming the 4-bit exponent is biased, what is the value of the following floating point number?

0 1101 10001110000

- How do you represent the number -2.875 in hex format, assuming a 4-bit biased representation and a normal mantissa?

- Defined by **IEEE** Std 754-1985
- Developed in response to **divergence** of representations
 - Portability issues for scientific code
- Now almost **universally** adopted
- Two representations
 - Single precision (32-bit)
 - Double precision (64-bit)

IEEE Floating Point Format

single: 8 bits
double: 11 bits

single: 23 bits
double: 52 bits



$$x = (-1)^S \times (1 + \text{Fraction}) \times 2^{(\text{Exponent} - \text{Bias})}$$

- S: sign bit (0 $\Rightarrow$ non-negative, 1 $\Rightarrow$ negative)
- Normalize significand: $1.0 \leq |\text{significand}| < 2.0$
 - Always has a leading pre-binary-point 1 bit, so no need to represent it explicitly (hidden bit)
 - Significand is Fraction with the “1.” restored (Unsigned)
- Exponent: **excess** representation: actual exponent + **Bias**
 - Ensures exponent is unsigned
 - Single: Bias = 127; Double: Bias = 1023

$$\text{Bias} = (2^{E-1} - 1)$$

Biased Exponent

Decimal Exponent	Signed-2's Complement	Biased Notation (Excess-16)	Decimal Value of Biased Notation
15	01111	11111	31
14	01110	11110	30
...	...	...	...
1	00001	10001	17
0	00000	10000	16 (bias)
-1	11111	01111	15
...	...	...	...
-15	10001	00001	1
-16	10000	00000	0

- Exponents 00000000 and 11111111 **reserved**
- Smallest value
 - Exponent: 00000001
 $\Rightarrow$ actual exponent = $1 - 127 = -126$
 - Fraction: 000...00 $\Rightarrow$ significand = 1.0
 - $\pm 1.0 \times 2^{-126} \approx \pm 1.2 \times 10^{-38}$
- Largest value
 - exponent: 11111110
 $\Rightarrow$ actual exponent = $254 - 127 = +127$
 - Fraction: 111...11 $\Rightarrow$ significand ≈ 2.0
 - $\pm 2.0 \times 2^{+127} \approx \pm 3.4 \times 10^{+38}$

- Exponents 0000...00 and 1111...11 **reserved**

- Smallest value

- Exponent: 000000000001

- $\Rightarrow \text{actual exponent} = 1 - 1023 = -1022$

- Fraction: 000...00 $\Rightarrow \text{significand} = 1.0$

- $\pm 1.0 \times 2^{-1022} \approx \pm 2.2 \times 10^{-308}$

- Largest value

- Exponent: 11111111110

- $\Rightarrow \text{actual exponent} = 2046 - 1023 = +1023$

- Fraction: 111...11 $\Rightarrow \text{significand} \approx 2.0$

- $\pm 2.0 \times 2^{+1023} \approx \pm 1.8 \times 10^{+308}$

- Represent -0.75
 - $-0.75 = (-1)^1 \times 1.1_2 \times 2^{-1}$
 - $S = 1$
 - Fraction = $1000...00_2$
 - Exponent = $-1 + \text{Bias}$
 - Single: $-1 + 127 = 126 = 01111110_2$
 - Double: $-1 + 1023 = 1022 = 01111111110_2$
- Single: $10111111101000...00$
- Double: $10111111111101000...00$

- What number is represented by the single-precision float

11000000101000...00

- $S = 1$
- Fraction = $01000...00_2$
- Exponent = $10000001_2 = 129$
- $$\begin{aligned} x &= (-1)^1 \times (1 + 01_2) \times 2^{(129 - 127)} \\ &= (-1) \times 1.25 \times 2^2 \\ &= -5.0 \end{aligned}$$

- Display the number -2.3 in single precision and IEEE standard Hex.

- **Exponent** = 000...0 $\Rightarrow$ hidden bit is 0

$$x = (-1)^s \times (0 + \text{Fraction}) \times 2^{-\text{Bias}}$$

- Smaller than normal numbers
 - allow for gradual underflow, with diminishing precision
- Denormal with fraction = 000...0

$$x = (-1)^s \times (0 + 0) \times 2^{-\text{Bias}} = \pm 0.0$$

Two representations
of 0.0!

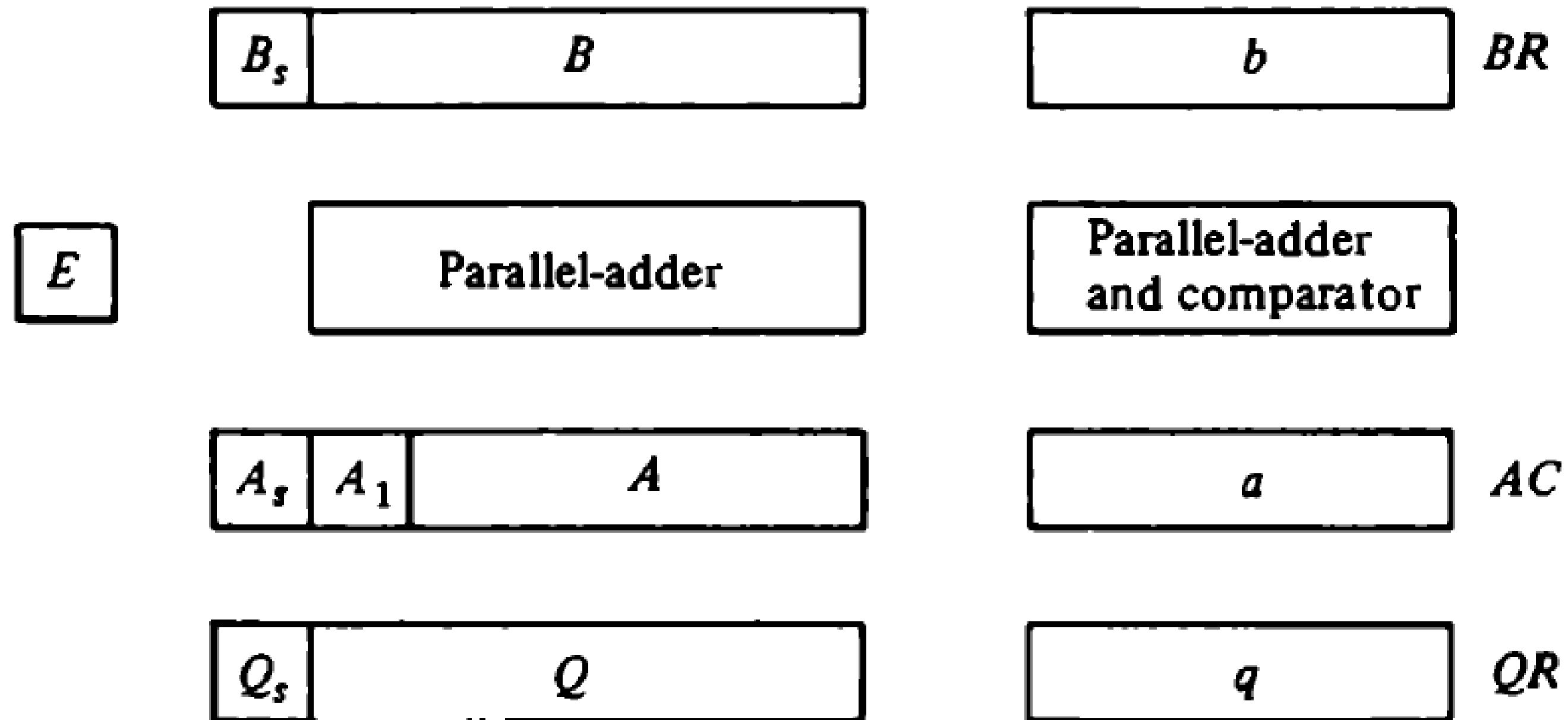


- Exponent = 111...1, Fraction = 000...0
 - $\pm$ Infinity
 - Can be used in subsequent calculations, avoiding need for overflow check
- Exponent = 111...1, Fraction $\neq$ 000...0
 - Not-a-Number (NaN)
 - Indicates illegal or undefined result
 - e.g., 0.0 / 0.0
 - Can be used in subsequent calculations

Single precision		Double precision		Object represented
Exponent	Fraction	Exponent	Fraction	
0	0	0	0	0
0	Nonzero	0	Nonzero	$\pm$ denormalized number
1–254	Anything	1–2046	Anything	$\pm$ floating-point number
255	0	2047	0	$\pm$ infinity
255	Nonzero	2047	Nonzero	NaN (Not a Number)

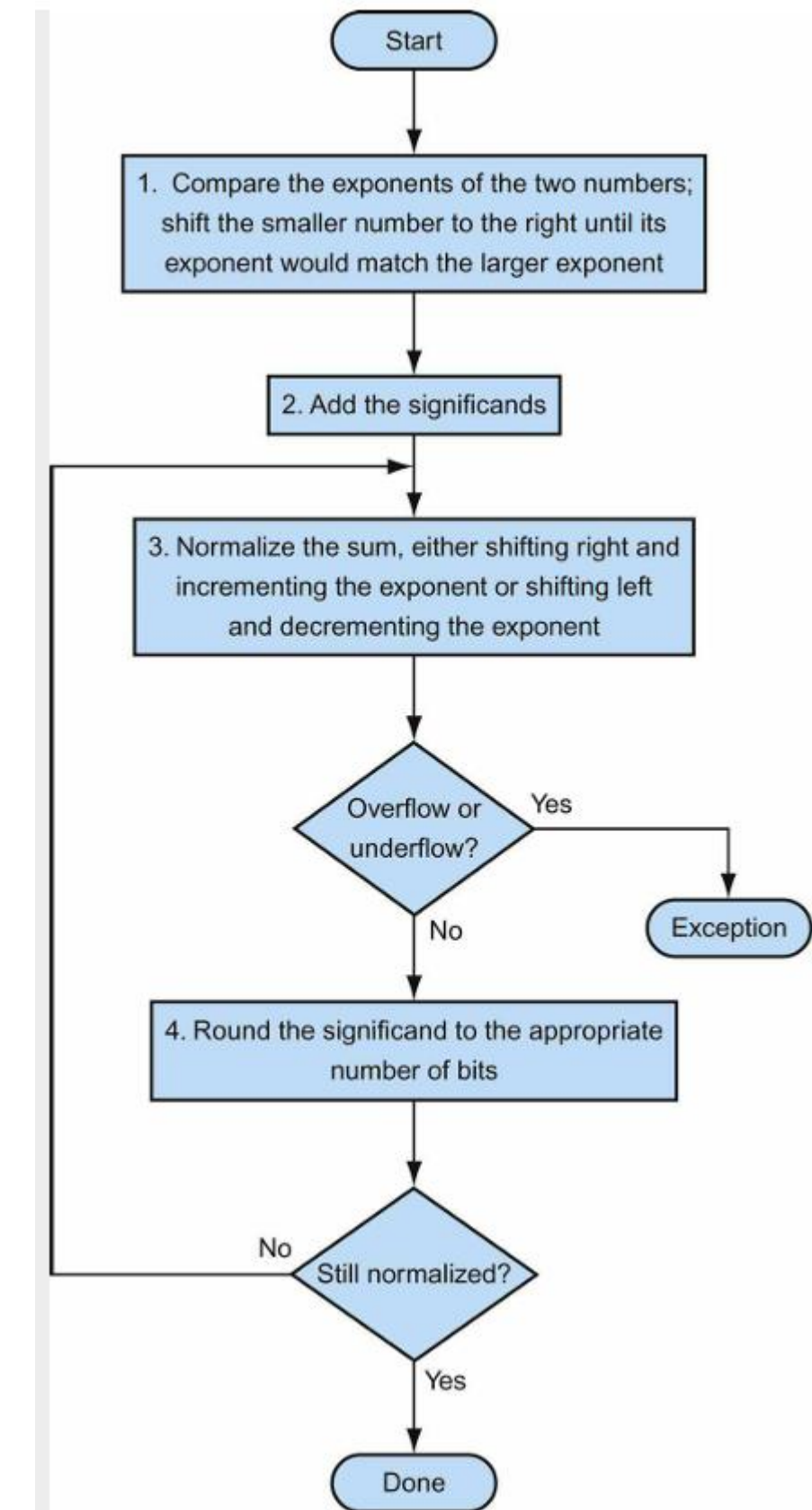
FIGURE 3.13 IEEE 754 encoding of floating-point numbers. A separate sign bit determines the sign. Denormalized numbers are described in the *Elaboration* on page 222. This information is also found in Column 4 of the MIPS Reference Data Card at the front of this book.

Figure 10-14 Registers for floating-point arithmetic operations.



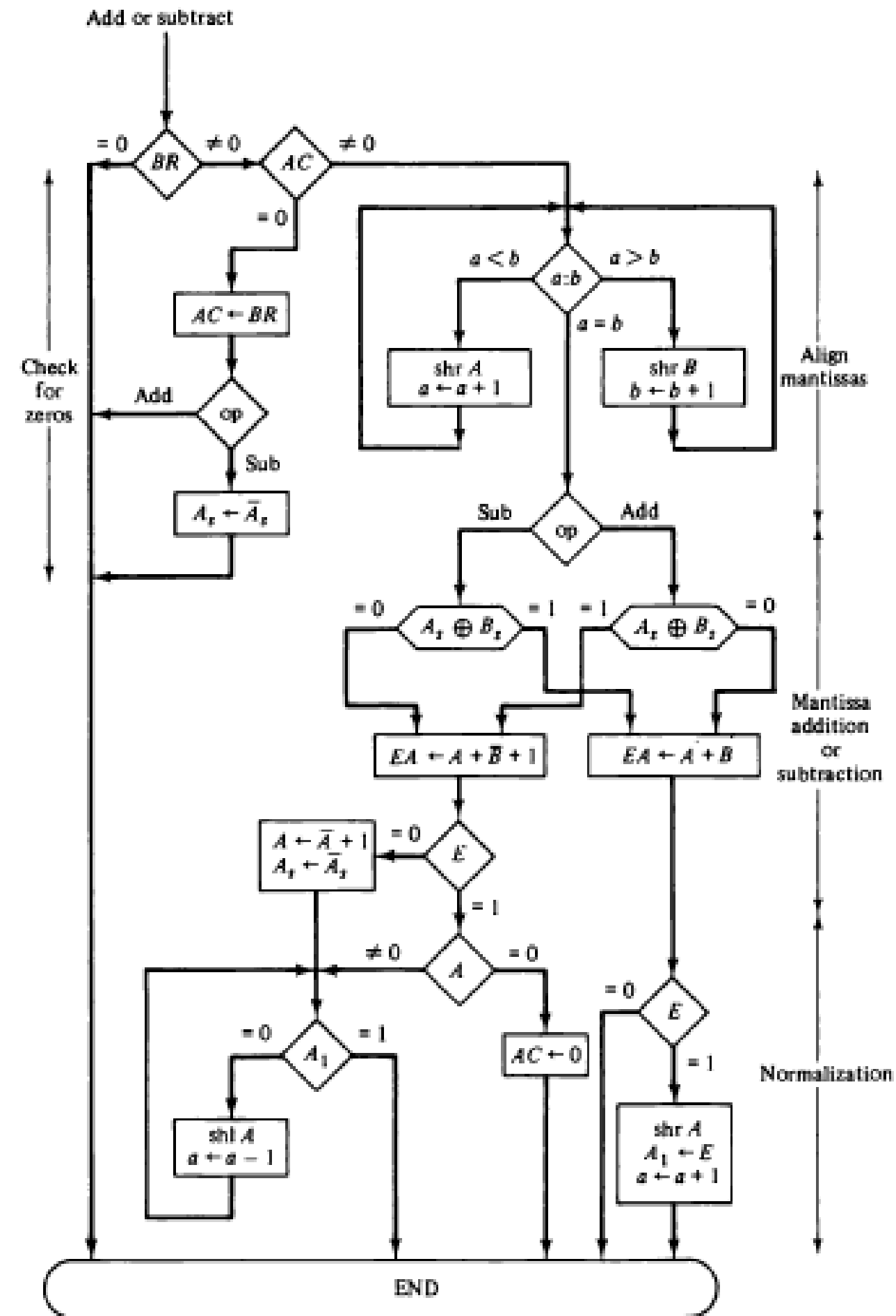
Floating-Point Addition

- Consider a 4-digit decimal example
 - $9.999 \times 10^1 + 1.610 \times 10^{-1}$
- 1. **Align** decimal points
 - Shift number with smaller exponent
 - $9.999 \times 10^1 + 0.016 \times 10^1$
- 2. **Add** significands
 - $9.999 \times 10^1 + 0.016 \times 10^1 = 10.015 \times 10^1$
- 3. **Normalize** result & check for **over/underflow**
 - 1.0015×10^2
- 4. **Round** and renormalize if necessary
 - 1.002×10^2



- Now consider a 4-digit binary example
 - $1.000_2 \times 2^{-1} + -1.110_2 \times 2^{-2}$ ($0.5 + -0.4375$)
- 1. **Align** binary points
 - Shift number with smaller exponent
 - $1.000_2 \times 2^{-1} + -0.111_2 \times 2^{-1}$
- 2. **Add** significands
 - $1.000_2 \times 2^{-1} + -0.111_2 \times 2^{-1} = 0.001_2 \times 2^{-1}$
- 3. **Normalize** result & check for over/underflow
 - $1.000_2 \times 2^{-4}$, with no over/underflow
- 4. **Round** and renormalize if necessary
 - $1.000_2 \times 2^{-4}$ (no change) = 0.0625

Floating-Point Addition

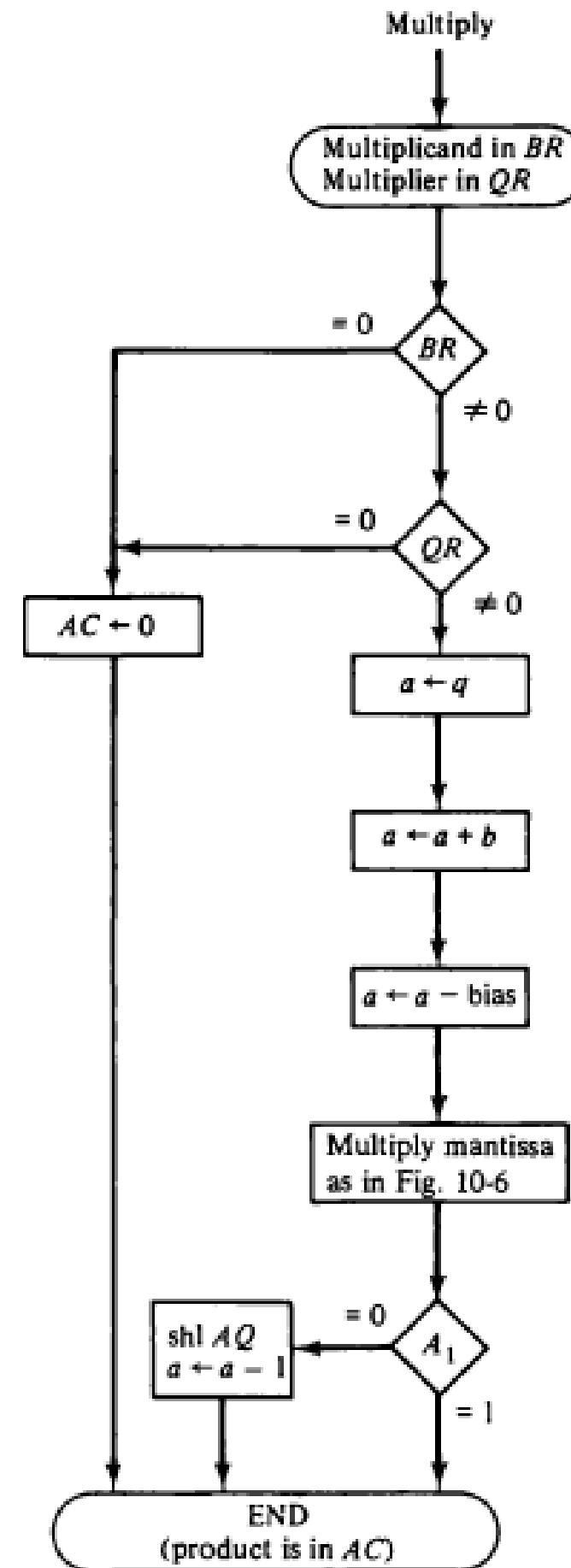


- Consider a **4-digit** decimal example
 - $1.110 \times 10^{10} \times 9.200 \times 10^{-5}$
- 1. **Add** exponents
 - For biased exponents, subtract bias from sum
 - New exponent = $10 + -5 = 5$
- 2. **Multiply** significands
 - $1.110 \times 9.200 = 10.212 \Rightarrow 10.212 \times 10^5$
- 3. **Normalize** result & check for over/underflow
 - 1.0212×10^6
- 4. **Round** and **renormalize** if necessary
 - 1.021×10^6
- 5. **Determine sign** of result from signs of operands
 - $+1.021 \times 10^6$

- Now consider a **4-digit** binary example
 - $1.000_2 \times 2^{-1} \times -1.110_2 \times 2^{-2}$ (0.5×-0.4375)
- 1. **Add** exponents
 - Unbiased: $-1 + -2 = -3$
 - Biased: $(-1 + 127) + (-2 + 127) = -3 + 254 - 127 = -3 + 127$
- 2. **Multiply** significands
 - $1.000_2 \times 1.110_2 = 1.110_2 \Rightarrow 1.110_2 \times 2^{-3}$
- 3. **Normalize** result & check for over/underflow
 - $1.110_2 \times 2^{-3}$ (no change) with no over/underflow
- 4. **Round** and **renormalize** if necessary
 - $1.110_2 \times 2^{-3}$ (no change)
- 5. **Determine sign**: $+ve \times -ve \Rightarrow -ve$
 - $-1.110_2 \times 2^{-3} = -0.21875$

Floating-Point Multiplication

Figure 10-16 Multiplication of floating-point numbers.



Floating-Point Division

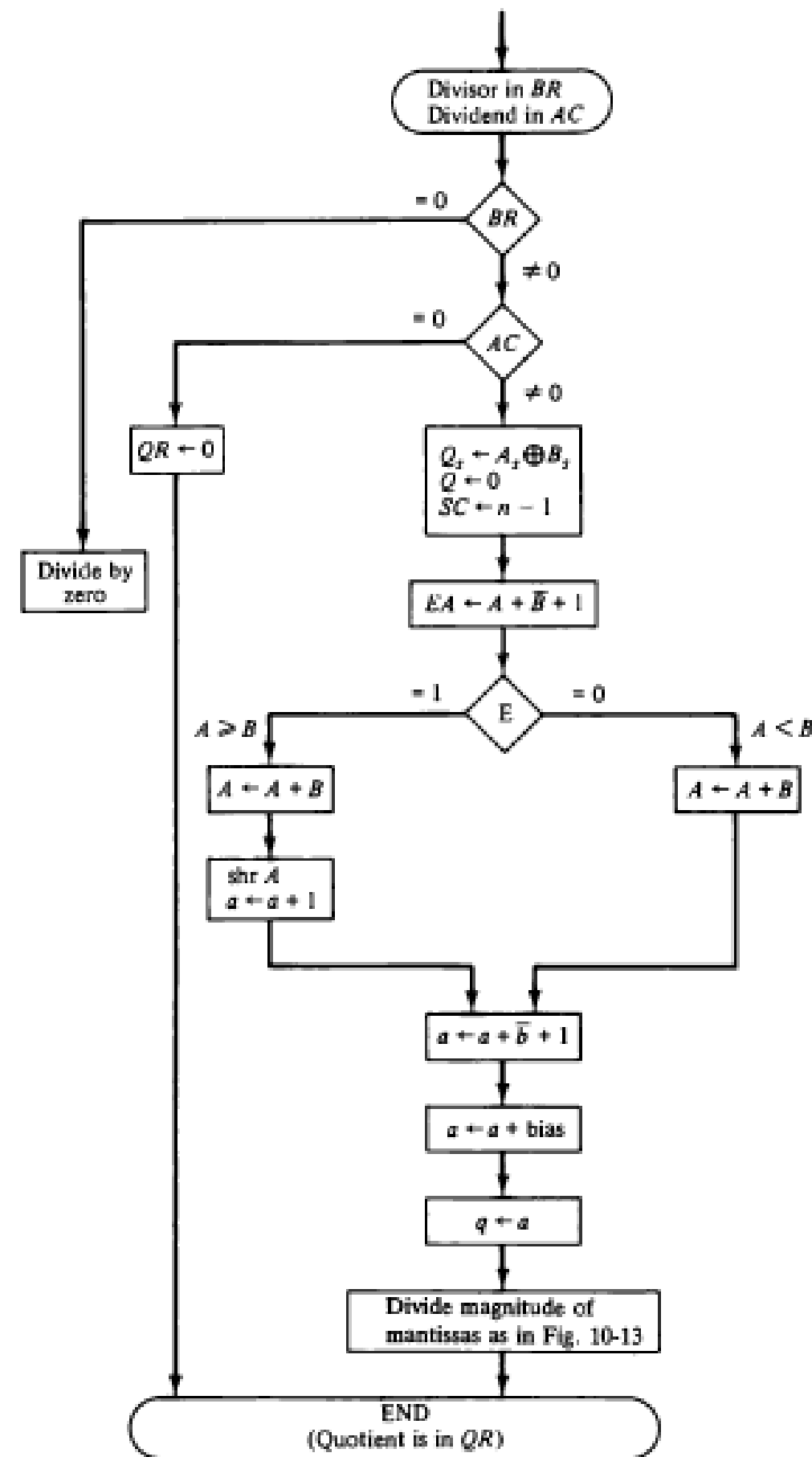


Figure 10-17 Division of floating-point numbers.

Example 1

یک سیستم نمایش اعداد ممیز شناور را در نظر بگیرید که بخش‌های مانتیس ۴ بیت، توان ۳ بیت و علامت یک بیت باشند (مطابق جدول زیر). نمایش توان biased-4 و نمایش مانتیس $\text{implicit one representation}$ فرض شده است. نمایش عدد دهدهی $+0.75$ در این صورت کدام است؟ (گزینه‌ها در مبنای ۱۶ هستند).

مانتیس	توان	علامت
۴ bit	۳ bit	۱ bit

(۱) ۳۱

(۲) ۳۸

(۳) ۴۳

(۴) ۴۵

Example 2

چه تعداد عملیات شیفت به راست در الگوریتم ضرب Booth برای انجام عملیات ضرب $(-13) \times (-9)$ ، مورد نیاز است؟

(۲) ۶

(۱) ۷

(۴) ۴

(۳) ۵

Example 3

اگر یک کامپیوتر از نمایش اعداد مکمل 2 و ثبات‌های 12 بیتی استفاده کند، دامنه نمایش اعداد صحیح در آن کدام است؟

(۱) از -2048 تا $+2047$

(۲) از -2047 تا $+2047$

(۳) از -4096 تا $+4095$

(۴) از -4095 تا $+4095$

Example 4

در یک پردازنده ۱۶ بیتی که محاسبات را به صورت مکمل دو انجام می دهد، سه بیت ثبات وضعیت به نام های S (Sign)، C (Carry out) و O (Overflow) وجود دارند. بعد از انجام عمل جمع دو عدد $F3E2$ و $EA29$ (اعداد در مبنای ۱۶ نمایش داده شده اند)، مقدار بیت های ثبات وضعیت کدام است؟

$$(2) \quad O = 0, C = 0, S = 1$$

$$(1) \quad O = 1, C = 1, S = 0$$

$$(4) \quad O = 1, C = 1, S = 1$$

$$(3) \quad O = 0, C = 1, S = 1$$

Example 5

یک سیستم نمایش اعداد ممیز شناور را با مشخصات ۱ بیت برای علامت، ۵ بیت برای توان و ۱۰ بیت برای مانتیس در نظر بگیرید. برای نمایش مانتیس از روش **Explicit One** و برای نمایش توان از روش **Biased-۱۶** استفاده می‌کنیم. اعداد به شکل هنجار شده (Normalized) با یک رقم صحیح و بقیه اعشاری دودویی نمایش داده می‌شوند. بزرگترین عدد مثبت قابل نمایش در این سیستم کدام است؟

$$(۱) \quad 2^{22} - 2^{23}$$

$$(۲) \quad 2^{22} - 2^{23}$$

$$(۳) \quad 2^{12} - 2^5$$

$$(۴) \quad 2^{16} - 2^6$$



Thanks
